

The Walks That Remember the Cycles

A machine-checked sharp gap law between the matching polynomial
and the non-backtracking spectrum in Lean 4

Carles Marín*

Independent researcher
karlesmarin@gmail.com

June 2026

Abstract

I present a machine-checked, **sorry-free** proof, in Lean 4 over Mathlib, of the *sharp trace-formula gap law* for a finite simple graph of girth g : for every $1 \leq k \leq g + 1$,

$$\mathrm{tr} A^k - p_k = \mathrm{tr} B^k,$$

where A is the adjacency matrix, B the Hashimoto non-backtracking operator, and $p_k = \sum_i \theta_i^k$ the power sums of the roots of the matching polynomial. Below the girth both sides vanish; at $k \in \{g, g + 1\}$ both sides count the rooted traversals of the shortest cycles, $2k c_k$; and the window is sharp—the law fails at $k = g + 2$ on every graph we tested, including all 12 064 connected cyclic graphs on at most 8 vertices. The proof is assembled from pieces of independent use: a cycle-extraction lemma for non-backtracking chains; a parity lemma for edge incidences of *arbitrary* closed walks (Mathlib’s version carries a trail hypothesis its own proof does not use); an exact incidence count for cycles; and a structure theorem—in the window, the closed non-backtracking walks, the rooted cycle traversals, and the closed walks that fail to lift to Godsil’s path tree are the *same* family, counted three ways. The headline theorem depends only on the three standard axioms. I am exact about the boundary: the mathematics is classical (Ihara, Hashimoto, Bass, Godsil; the law itself is folklore-adjacent and known to coding theorists in other clothing); the contribution is the formalization—to the best of my knowledge the first machine-checked bridge between the matching polynomial and the non-backtracking spectrum in any proof assistant—and the sharp window is stated and proved as such. A companion paper applies the law, as a certified census of short cycles, to the LDPC codes deployed in the IEEE 802.11n (WiFi) standard.

Reader’s map. If you read only one thing, read Theorem 1 and Figure 2—the statement, and the picture of the law holding on its window and breaking, spectacularly, just past it. The proof is a single chain (Figure 1); every link has its own section and its own Lean name (Table 2). The honest limits are in Section 8.

1 Introduction: a trace formula comes down a ladder

Begin with the anomaly. Take the Petersen graph and run two censuses that have no obvious business agreeing: count all closed walks of length k and subtract those that lift to Godsil’s path tree

*Formalized with AI assistance (Claude, Anthropic); the mathematics and all claims are the author’s responsibility. The methodology is discussed in Section 8.

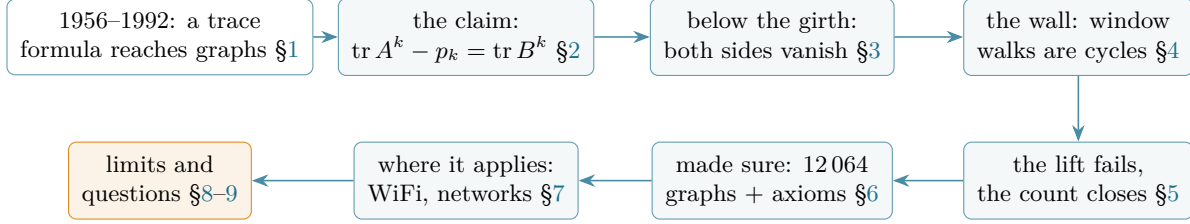


Figure 1: The reader’s map: the claim, the three stones of its proof, the checks, the uses, and what remains.

$(\text{tr } A^k - p_k)$; separately, count the closed walks that never immediately backtrack ($\text{tr } B^k$). At $k = 5$ both give 120. At $k = 6$, both give 120 again. At $k = 7$ the first census jumps to 1680—and the second collapses to 0, because the Petersen graph has no 7-cycles at all (Figure 2). Two censuses in lockstep, then a cliff. The lockstep is a theorem; the cliff is its sharpness; this paper machine-checks both. Where does such a pair of censuses come from?

In 1956 Selberg proved a formula that holds two ways of hearing a hyperbolic surface against each other: on one side the spectrum of the Laplacian, on the other the lengths of the closed geodesics [6]. Ten years later Ihara, working p -adically, found the discrete shadow of that formula: a zeta function for a graph, built from its closed geodesics—the closed walks that never immediately backtrack [7]. What had been analysis on a surface became linear algebra on a finite object: Hashimoto encoded the non-backtracking condition as a 0/1 matrix B on directed edges [11], and Bass proved that its characteristic polynomial factors through the adjacency matrix [12]—the graph-theoretic Selberg identity, formalized in Part III of this series [29].

A trace formula needs a second side. On a surface it is the Plancherel term, the contribution of the universal cover; on a graph the universal cover is a tree, and the object that hears the tree is the *matching polynomial*: Godsil proved in 1981 that the power sums of its roots count the closed walks that lift to the *path tree*—the “tree-like” walks [9], a theorem this series formalized in Part IV [31], resting on the real-rootedness of Heilmann and Lieb [8] formalized in Part II [28].

So by Part V the series had built, separately, the two sides of a trace formula: the tree side (p_k , matching polynomial, path tree) and the geodesic side ($\text{tr } B^k$, non-backtracking operator, Ihara zeta). Each had its own paper, its own verifier runs, its own Zenodo record. They had never met inside the proof assistant.

This paper is the meeting. The bridge is a subtraction:

$$\text{tr } A^k - p_k,$$

all closed walks minus the tree-like ones—the walks the tree cannot explain. The theorem is that on a sharp window, $1 \leq k \leq g + 1$ with g the girth, this remainder *is* the non-backtracking trace $\text{tr } B^k$: what walks forget about the tree, they remember about the cycles. The window is exact. One step past it, at $k = g + 2$, the first walk appears that wraps a cycle and then wanders—and the two sides come apart, on every graph we tested (Figure 2). The payoff of the necklace is that this is the first theorem of the series that *consumes both strands at once*: its Lean proof imports the path-tree machinery of Part IV and the Hashimoto operator of Part III in the same file, and neither side could be stated without the other’s vocabulary.

2 The claim: what exactly is being subtracted?

Throughout, G is a finite simple graph on vertex set V , A its adjacency matrix, and g its girth (the length of a shortest cycle; $g \geq 3$). The matching polynomial is

$$\mu_G(x) = \sum_{j \geq 0} (-1)^j m_j x^{n-2j},$$

with m_j the number of j -edge matchings, and $p_k = \sum_i \theta_i^k$ the k -th power sum of its roots (over $\mathbb{C}$; the roots are in fact real [8, 28]). The Hashimoto operator B acts on the $2|E|$ darts (ordered edges): $B_{d,e} = 1$ exactly when e continues d without reversing it (snd $d = \text{fst } e$ and $e \neq \bar{d}$) [11, 29].

Theorem 1 (the sharp gap law; `trace_sub_matchingPowerSum_eq_trace_hashimoto`). *For every k with $1 \leq k \leq g + 1$,*

$$\text{tr } A^k - p_k = \text{tr } B^k.$$

Below the girth ($k < g$) both sides are 0. At $k \in \{g, g + 1\}$ both sides equal $2k c_k$, where c_k is the number of k -cycles of G .

The Lean statement is exactly this, over $\mathbb{C}$, with p_k the literal `Multiset.sum` of k -th powers of the roots of `matchingPoly` mapped into $\mathbb{C}$; the integer form `treeLikeGap_eq_trace_hashimoto` states the same identity over $\mathbb{Z}$ with p_k replaced by Godsil’s tree-like walk count, and the two are joined by Part IV’s moment theorem. No regularity, connectivity or bipartiteness is assumed. The artifact itself, verbatim:

```
theorem trace_sub_matchingPowerSum_eq_trace_hashimoto
  [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
  {k : ℕ} (hk : 0 < k) (hwin : (k : ℕ_infinity) <= G.egirth + 1) :
  ((G.adjMatrix N ^ k).trace : ℂ) - G.matchingPowerSum k
  = ((G.hashimoto C) ^ k).trace
```

One example fits in the head. On the triangle K_3 at $k = g = 3$: there are six closed walks of length 3 (three base points, two directions), so $\text{tr } A^3 = 6$; none of them is tree-like—a walk around a triangle does not lift to a tree—and indeed $p_3 = 0$, since $\mu_{K_3} = x^3 - 3x$ has roots $0, \pm\sqrt{3}$ whose cubes cancel; and every one of the six is non-backtracking, so $\text{tr } B^3 = 6$. The law reads $6 - 0 = 6$, and each number is checkable without paper. The theorem is that this never fails until $k = g + 2$.

Three remarks before the proof. First, the subtraction is meaningful because of Part IV: p_k is a count of walks (the tree-like ones), so the left side is a difference of two censuses of the same population. Second, the right side is also a census: $\text{tr } B^k$ counts the closed non-backtracking dart walks of length k (`trace_hashimoto_pow`, Part V of the series mapped this in the Jacobi–Newton paper [30]). The theorem is therefore a *bijection of censuses*, and that is how it is proved. Third, the window matters: for $k \geq g + 2$ the law is not merely unproved but *false*, and visibly so (Figure 2; Section 6).

3 Stone A: why is there nothing below the girth?

Serre’s *Trees* taught a generation the slogan: below its girth, a graph cannot tell itself apart from its universal cover [4]. A formal proof cannot wave at a cover; it needs the slogan as a finite witness.

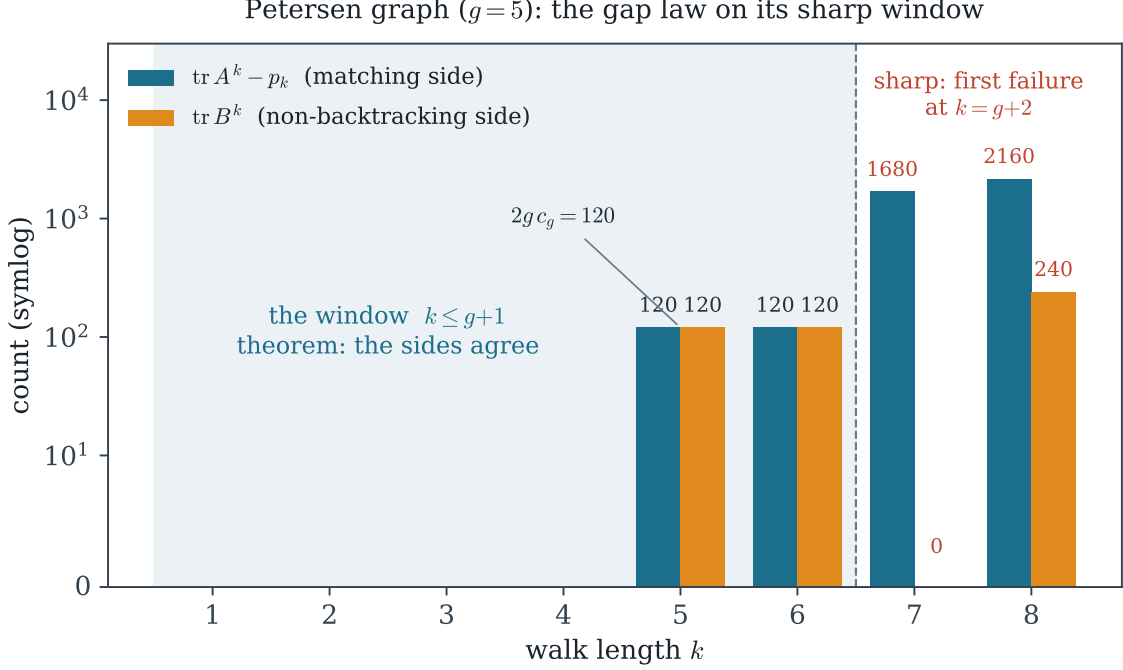


Figure 2: The gap law on the Petersen graph ($g=5$), in exact integer arithmetic. On the window $k \leq 6$ the two censuses agree—both are 0 below the girth and $120 = 2 \cdot 5 \cdot 12$ at $k=5$ (Petersen has twelve pentagons), $120 = 2 \cdot 6 \cdot 10$ at $k=6$ (ten hexagons). At $k=7$ the law breaks as violently as it can: the left side jumps to 1680 while the right side is 0—the Petersen graph famously has no 7-cycles, so there are no closed non-backtracking walks of length 7 at all, while 1680 closed walks of length 7 already fail to be tree-like (a pentagon plus a dangling edge). Sharpness is not an artifact of one graph: it holds on all 12064 connected cyclic graphs on ≤ 8 vertices (Section 6).

Below the girth the law says $0 = 0$, but neither 0 is free. The right side asks: why is there no closed non-backtracking walk shorter than the shortest cycle? Classically one waves at the universal cover; the formal proof wants a finite witness, and the witness is a *cycle extracted from the walk itself*.

Lemma 1 (cycle extraction; `Walk.exists_isCycle_of_nbChain_of_not_nodup`). *Let w be a walk whose darts form a non-backtracking chain and whose vertex support repeats a vertex. Then G contains a cycle of length at most the length of w , supported on the edges of w .*

The proof is a structural induction with no rotation anywhere—rotating a walk would break the *linear* non-backtracking chain at the seam, an obstruction that silently corrupts informal proofs. If the tail of the walk already repeats a vertex, induct. Otherwise the head vertex u re-enters the tail; the prefix up to that re-entry is a path (`IsPath.takeUntil`), and the closing edge either avoids the path—then `cons_isCycle_iff` hands us the cycle—or lies on it, which pins the path to the single reversed dart of the entering edge and contradicts the chain at its first link. Stone A follows by a pigeonhole: a closed non-backtracking dart walk of length k presents $k+1$ darts whose last repeats its first, hence at most k distinct edges; the extracted cycle needs at least g of them; so $k \geq g$ (`relWalks_nbRel_closed_eq_empty`, `trace_hashimoto_pow_eq_zero_of_lt_egirth`). The left side's 0 is Part IV's below-girth theorem (every short closed walk lifts to the path tree). The gap law below the girth is then a subtraction of zeros—but each zero is a theorem.

k	$\text{tr } A^k$	p_k	$\text{tr } A^k - p_k$	$\text{tr } B^k$
1	0	0	0	0
2	30	30	0	0
3	0	0	0	0
4	150	150	0	0
5	120	0	120	120
6	990	870	120	120
7	1680	0	1680	0
8	7590	5430	2160	240

Table 1: The Petersen data behind Figure 2: spectra give $\text{tr } A^k = 3^k + 5 + 4(-2)^k$; p_k by Newton’s identities from $\mu = x^{10} - 15x^8 + 75x^6 - 145x^4 + 90x^2 - 6$; $\text{tr } B^k$ by the Ihara–Bass per-eigenvalue recurrence $s_k = \lambda s_{k-1} - 2s_{k-2}$ plus $5(1 + (-1)^k)$. The window (bold) and the first two failures (amber). Every entry is exact.

4 The wall: why is a window walk forced to be a cycle?

The oldest blade in the book is Euler’s, Königsberg, 1736: every entrance needs an exit [1]. Three centuries later the same parity argument decides the case that modern hypotheses cannot reach.

The content of the law lives at $k \in \{g, g + 1\}$, and it is a *structure* theorem:

Theorem 2 (window rigidity; `Walk.isCycle_of_nbChain_window`). *A closed non-backtracking walk of length k with $1 \leq k \leq g + 1$ is a cycle.*

What surprised us in the formalization is how little “non-backtracking” the proof ultimately uses: the chain hypothesis enters *only* through Lemma 1. The extracted cycle C satisfies $g \leq |C| \leq k \leq g + 1$, leaving exactly two cases, and both die by counting edge incidences—parity and degree, nothing else.

The parity blade. For any walk, count the edge slots incident to a vertex x . Mathlib proves the parity of this count for *trails* (`IsTrail.even_countP_edges_iff`), but its own induction never uses the trail hypothesis except to feed itself; removing it (`Walk.even_countP_edges_iff'`) gives: in a *closed* walk, every vertex has even incidence count. This matters because the walks in case $|C| = k - 1$ need not be trails.

The degree blade. A cycle meets each vertex of its support in exactly two edge slots (`Walk.countP_edges_of_isCycle`), a transfer of Mathlib’s `ncard_neighborSet_toSubgraph_eq_two` along an explicit two-sided bijection $e \mapsto$ “the other endpoint”.

Case $|C| = k - 1$. The walk’s edge multiset is the cycle’s edges plus one extra slot f (a multiset surgery isolates f). Any endpoint x of f then has incidence $2 + 1 = 3$ or $0 + 1 = 1$ —odd either way, against the parity blade. No such walk exists (`Walk.not_closed_of_isCycle_edges_add_one`).

Case $|C| = k$. The walk’s edges are a permutation of the cycle’s, so the walk is a trail. If its support repeated an interior vertex x , rotate the walk to base x (legitimate here—trails survive rotation, and we no longer need the chain): x now sits at three pairwise-distinct slots $\{0, m - 1, m\}$, giving three *distinct* edges at x , against the degree blade’s two. So the support is simple and the walk is the cycle (`Walk.isCycle_of_edges_perm`).

Composing the two cases with Lemma 1 proves Theorem 2. The window hypothesis is used exactly once—to squeeze $|C|$ into $\{k - 1, k\}$ —and this is precisely what fails at $k = g + 2$: a g -cycle plus a 2-step excursion becomes available, parity is satisfied ($2 + 2$), and the law breaks (Figures 2 and 3).

Why the window is sharp: the first walk that one census sees and the other does not

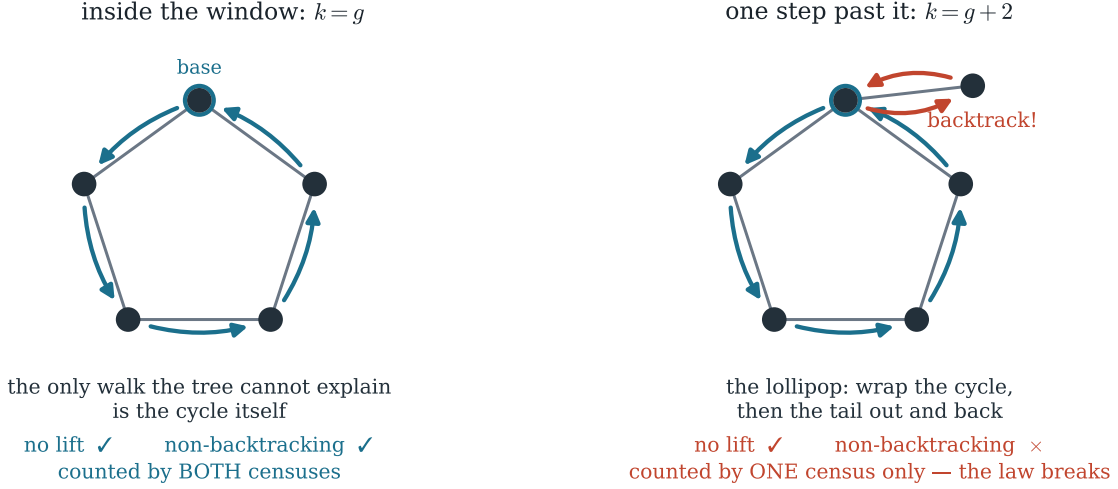


Figure 3: The structure theorem in one picture. Left: inside the window the only closed walk the path tree cannot explain is a cycle traversal, and a cycle never backtracks—the two censuses count the same object. Right: at $k = g + 2$ the first *lollipop* appears: it wraps the pentagon (so it does not lift; the matching census counts it) but its tail turns back on itself (so the non-backtracking census rejects it). One walk, one census, no law.

5 The other side, and the count that closes the bridge

Counting by bijection is combinatorics’ oldest trust protocol: Prüfer certified Cayley’s n^{n-2} by exhibiting the correspondence, one tree at a time [2]. The gap law closes the same way, one walk at a time.

The left census needs the mirror statement: in the window, a closed walk fails to lift to the path tree *exactly when* it is a cycle. One direction is a pleasant computation in the lift itself: along a cycle the stack only ever extends, and at the closing step the root is in the stack but is not its penultimate entry, so the retreat fails (`Walk.not_isTreeLike_of_isCycle`). The other direction needed, on paper, a second wall—and dissolved instead into pieces already proved: if a window walk does not lift, its edge support contains a cycle (the contrapositive of Part IV’s `isTreeLike_of_acyclic`); mapping that cycle into G lands exactly in the two cases of Section 4, whose conclusions now read “the walk is that cycle” or “false” (`Walk.isCycle_or_isTreeLike_window`).

So in the window three families coincide:

$$\{\text{closed NB dart walks}\} = \{\text{rooted cycle traversals}\} = \{\text{closed walks that do not lift}\},$$

and the gap law is the statement that the first and third have the same cardinality. The final bijection (`sum_card_not_treeLike_eq_sum_card_relWalks`) sends a non-lifting walk—now known to be a cycle—to its dart list closed by the repeat of its first dart; injectivity rests on a small lemma of independent value, *a walk is determined by its darts* (`Walk.eq_of_darts_eq`); surjectivity reassembles the walk from the dart list with Mathlib’s `Walk.ofDarts`. Summing over roots and casting through $\mathbb{Z}$ gives the capstone, and Part IV’s moment theorem rewrites the tree-like count as p_k : Theorem 1.

Lean name	statement	where
<code>trace_sub_matchingPowerSum_eq_trace_hashimoto</code>	$\text{tr } A^k - p_k = \text{tr } B^k$ on the window	§2
<code>treeLikeGap_eq_trace_hashimoto</code>	the capstone over $\mathbb{Z}$	§2
<code>exists_isCycle_of_nbChain_of_not_nodup</code>	cycle extraction from an NB chain	§3
<code>trace_hashimoto_pow_eq_zero_of_lt_egirth</code>	$\text{tr } B^k = 0$ for $k < g$	§3
<code>even_countP_edges_iff'</code>	incidence parity, no trail hypothesis	§4
<code>countP_edges_of_isCycle</code>	a cycle has incidence count 2	§4
<code>not_closed_of_isCycle_edges_add_one</code>	no walk is a cycle plus one slot	§4
<code>isCycle_of_edges_perm</code>	a closed walk on a cycle's edges is the cycle	§4
<code>isCycle_of_nbChain_window</code>	window rigidity (Theorem 2)	§4
<code>not_isTreeLike_of_isCycle</code>	a cycle does not lift to the path tree	§5
<code>isCycle_or_isTreeLike_window</code>	the window dichotomy	§5
<code>eq_of_darts_eq</code>	a walk is determined by its darts	§5
<code>sum_card_not_treeLike_eq_sum_card_relWalks</code>	the bijection of censuses	§5

Table 2: The load-bearing results, all sorry-free in `Ihara/NbVanishing.lean` and `Ihara/GapWindow.lean` of the repository. Axioms of the capstone: `propext`, `Classical.choice`, `Quot.sound` only.

6 Made sure: twelve thousand graphs and three axioms

Numerical evidence is a compass, not a deed: Mertens' conjecture agreed with every computer ever pointed at it and fell anyway [5]. The two instruments here divide the labor in the opposite direction—the numerics choose the statement, the proof assistant owns it.

Formal proofs eliminate one kind of error; choosing the *right statement* requires another instrument, and the statement here—in particular the claim that the window is sharp—was locked numerically before a line of Lean was written, and re-locked after.

graph	g	c_g	$\text{gap}_g = \text{tr } B^g$	window verified	first failure
K_3	3	1	6	$k \leq 4$	$k = 5$
C_5	5	1	10	$k \leq 6$	$k = 7$
K_4	3	4	24	$k \leq 4$	$k = 5$
$K_{3,3}$	4	9	72	$k \leq 5$	$k = 6$
Q_3 (cube)	4	6	48	$k \leq 5$	$k = 6$
Petersen	5	12	120	$k \leq 6$	$k = 7$

exhaustive sweep: all 12 064 connected cyclic graphs, $4 \leq n \leq 8$ (Sage, `nauty_geng`):
0 violations on the window; the law failed at $k = g + 2$ on *every single graph*.

Table 3: Numerical locks. The named graphs are checked in exact arithmetic (`research/_tmp/traceformula_lock.py`); the sweep (`research/_tmp/gaplaw_sweep.py`) computes both sides independently— B as an explicit dart matrix, p_k by Newton from matching counts—and confirms that sharpness at $g + 2$ is universal in this range, not generic.

On the formal side: the development builds with Lean toolchain v4.30.0-rc2 against a pinned Mathlib (3054 jobs, exit 0), the two new files are warning-free, and `#print axioms` on every theorem of Table 2 reports `propext`, `Classical.choice`, `Quot.sound`—the three standard axioms—and nothing else. No `native_decide`, no `Decide`-bombs, no custom axioms.

Checking this paper is three commands:


```
git clone https://github.com/karlesmarin/godsil-gutman-lean
cd godsil-gutman-lean && lake exe cache get && lake build Ihara
echo 'import Ihara.GapWindow
      #print axioms
      SimpleGraph.trace_sub_matchingPowerSum_eq_trace_hashimoto' \
| lake env lean --stdin
```

7 Where it applies: from a trace identity to a WiFi chip

Gallager invented LDPC codes in 1962 and the world shelved them for thirty years [3]; when they returned (to WiFi, to 5G, to deep space), the girth of a graph had become an engineering quantity. That is where a trace identity meets a chip.

Certified cycle censuses. At $k = g$ the law is a formula for the number of shortest cycles, $c_g = (\text{tr } A^g - p_g)/2g$, computable from the spectrum and the first few matching counts. Short cycles in the Tanner graph of an LDPC code degrade belief-propagation decoding, and counting them is an established engineering task [22, 23]. The companion paper applies the (now theorem-backed) formula as a *certified census* to the four LDPC codes of IEEE 802.11n at $n = 648$: three mutually independent computations agree exactly on all four— $c_6 = 3942$ (rate $1/2$), $c_6 = 8046$ (rate $2/3$), $c_4 = 54$ (rate $3/4$, where the girth collapses to 4), $c_6 = 32\,346$ (rate $5/6$). The certification layer, not the counting speed, is the contribution there: the counting algorithms of [22, 23] are faster and reach further ($k \leq 2g - 2$), but carry no machine-checked guarantee.

The recipe: count your graph's shortest cycles in four lines

Given a graph with girth g (even case shown; m_j = number of j -matchings): (1) compute $\text{tr } A^g$ from the spectrum or by matrix powers; (2) count the small matchings, $m_1 = |E|$, $m_2 = \binom{m_1}{2} - \sum_v \binom{d_v}{2}$, ... up to $m_{g/2}$; (3) Newton: $p_2 = 2m_1$, $p_4 = 2m_1^2 - 4m_2$, $p_6 = 2m_1^3 - 6m_1m_2 + 6m_3$; (4) then $c_g = (\text{tr } A^g - p_g)/2g$, with a machine-checked guarantee. *Worked:* $K_{3,3}$ has $g = 4$, eigenvalues $\pm 3, 0^4$, so $\text{tr } A^4 = 162$; $m_1 = 9$, $m_2 = \binom{9}{2} - 6\binom{3}{2} = 18$, so $p_4 = 2 \cdot 81 - 4 \cdot 18 = 90$; hence $c_4 = (162 - 90)/8 = 9$: the nine quadrilaterals of $K_{3,3}$, certified.

Chemistry: the matching polynomial is measurable. In the Hückel theory of conjugated hydrocarbons, μ_G is the *reference polynomial*: the topological resonance energy of a molecule is the difference between the spectra of A (the π -system) and of μ_G (the same system with its cycles erased)—aromaticity as a spectral gap between a graph and its own tree shadow [16, 17]. The gap law is the walk-level identity beneath that chemistry: it says *exactly which* walk counts the two spectra share (all of them, up to $g + 1$) and where they first part. The k -th moments of benzene's π -system and of its reference polynomial agree until $k = 6$ wraps the hexagon—which is the aromatic sextet seen combinatorially.

Epidemics and percolation. The percolation threshold of a sparse network and the epidemic threshold of an SIR process are both governed by $1/\lambda_1(B)$, the leading eigenvalue of the non-backtracking operator [19]. Numerical estimates of $\lambda_1(B)$ on a large network are exactly the situation where certified low moments matter: $\text{tr } B^g = 2g c_g$ and $\text{tr } B^{g+1}$ pin the first nontrivial moments of the spectral distribution from quantities ($\text{tr } A^k$, small matchings) that are computed by independent, well-tested code paths.

Non-backtracking methods in network science. The operator B has become a standard tool: non-backtracking random walks mix faster [20], and the spectrum of B achieves the detectability threshold in community detection where adjacency methods fail [21]. The gap law gives the smallest

moments of B exactly, in terms of classical, well-understood quantities ($\text{tr } A^k$ and matchings)—the entry-level sanity checks for any numerical implementation of B .

The Ramanujan connection. Part I of this series formalized the band $2\sqrt{d-1}$ [27]; graphs achieving it (Ramanujan graphs, with the explicit constructions of Lubotzky–Phillips–Sarnak attaining girth $\sim \frac{4}{3} \log_{d-1} n$ [18]) have near-optimal girth, hence the *widest certified windows* for this law. Design guarantees few short cycles; the gap law counts exactly how few. The two wings—spectral bound and trace identity—are now both machine-checked, in the same repository.

8 The honest boundary

The mathematics is classical. The trace-formula reading of the matching polynomial versus the Ihara zeta is implicit in the literature around Godsil [9, 10], Bass [12], Stark–Terras [13] and Terras’s book [15]; coding theorists know the $k \leq g+1$ census in other clothing [23]. I claim no new mathematics. The contribution is the machine-checked bridge—and the discipline the formalization forced (the seam obstruction to rotation, the de-trailed parity lemma, the exact point where the window hypothesis enters), which is where the genuine novelty of this kind of work lives.

The window is $k \leq g+1$, full stop. Engineers count to $2g-2$ with correction terms [23, 24]; none of that is formalized here, and the law as stated is *false* beyond the window.

Prior formalizations. I was unable to locate any machine-checked statement connecting the matching polynomial to the non-backtracking operator, in any proof assistant; searches on 2026-06-11/12 over Mathlib, the Lean community repositories, Isabelle’s AFP and Coq packages found neither the law nor its ingredients (B itself enters Lean in Part III of this series). Such a claim can only ever be made to the best of one’s knowledge.

Methodology. The formalization was carried out with AI assistance (Claude, Anthropic) under the discipline used throughout this series: every statement locked by exact numerics before formalization; every theorem checked by `#print axioms`; every claimed “first” wheel-checked against the literature before being claimed. The mathematics and all errors are mine.

9 Open questions

1. *The corrected window.* For $g+2 \leq k \leq 2g-2$ the failure is structured: the defect $\text{tr } A^k - p_k - \text{tr } B^k$ counts lollipop walks, and closed-form corrections are known informally [23, 24]. Formalize the first correction term. What walk family does the $k = g+2$ defect count, exactly, in Lean’s vocabulary?
2. *From traces to geodesics.* $\text{tr } B^k$ counts *rooted* non-backtracking walks; the Ihara zeta wants their rotation classes (prime geodesics), a Möbius inversion away. The cyclic-orbit machinery is half-built in this repository (`MathlibPR/MatrixWalkCount.lean`); the weighted Burnside step is open. Can the Euler product itself be made a theorem?
3. *Upstreaming.* `eq_of_darts_eq`, the de-trailed parity lemma, the cycle incidence count and the directed walk-counting library are Mathlib-shaped. Which of these survive review, and in what generality?
4. *The window as an invariant.* The pair $(\text{tr } B^g, \text{tr } B^{g+1}) = (2g c_g, 2(g+1)c_{g+1})$ is a cheap, certified graph invariant. How much of the isomorphism class of small graphs does the certified window pin down—and where does it first fail to distinguish?

5. *One to keep.* A subtraction is a measurement: the gap law says a graph can count its own shortest cycles by forgetting its trees. The tree side (p_k) is the universal cover speaking; the cycle side ($\text{tr } B^k$) is the fundamental group answering. At what k —in general, not just past $g + 1$ —does a finite graph stop being explainable by its own universal cover?

Data and code availability

The Lean sources (`Ihara/NbVanishing.lean`, `Ihara/GapWindow.lean`, with the Part III–V files they import), the numerical locks and the figure scripts are in the public repository <https://github.com/karlesmarin/godsil-gutman-lean>. The companion applied paper (the certified LDPC census) is referenced there.

References

- [1] L. Euler, *Solutio problematis ad geometriam situs pertinentis*, Comment. Acad. Sci. Petrop. **8** (1736) 128–140.
- [2] H. Prüfer, *Neuer Beweis eines Satzes über Permutationen*, Arch. Math. Phys. **27** (1918) 742–744.
- [3] R. G. Gallager, *Low-density parity-check codes*, IRE Trans. Inform. Theory **8** (1962) 21–28.
- [4] J.-P. Serre, *Trees*, Springer, 1980 (French original *Arbres, amalgames, SL_2* , 1977).
- [5] A. M. Odlyzko, H. J. J. te Riele, *Disproof of the Mertens conjecture*, J. Reine Angew. Math. **357** (1985) 138–160.
- [6] A. Selberg, *Harmonic analysis and discontinuous groups in weakly symmetric Riemannian spaces with applications to Dirichlet series*, J. Indian Math. Soc. **20** (1956) 47–87.
- [7] Y. Ihara, *On discrete subgroups of the two by two projective linear group over p -adic fields*, J. Math. Soc. Japan **18** (1966) 219–235.
- [8] O. J. Heilmann, E. H. Lieb, *Theory of monomer-dimer systems*, Comm. Math. Phys. **25** (1972) 190–232.
- [9] C. D. Godsil, *Matchings and walks in graphs*, J. Graph Theory **5** (1981) 285–297.
- [10] C. D. Godsil, *Algebraic Combinatorics*, Chapman & Hall, 1993.
- [11] K. Hashimoto, *Zeta functions of finite graphs and representations of p -adic groups*, Adv. Stud. Pure Math. **15** (1989) 211–280.
- [12] H. Bass, *The Ihara–Selberg zeta function of a tree lattice*, Internat. J. Math. **3** (1992) 717–797.
- [13] H. M. Stark, A. Terras, *Zeta functions of finite graphs and coverings*, Adv. Math. **121** (1996) 124–165.
- [14] M. Kotani, T. Sunada, *Zeta functions of finite graphs*, J. Math. Sci. Univ. Tokyo **7** (2000) 7–25.
- [15] A. Terras, *Zeta Functions of Graphs: A Stroll through the Garden*, Cambridge Univ. Press, 2011.
- [16] J. Aihara, *A new definition of Dewar-type resonance energies*, J. Amer. Chem. Soc. **98** (1976) 2750–2758.
- [17] I. Gutman, *The matching polynomial*, MATCH Commun. Math. Comput. Chem. **6** (1979) 75–91.
- [18] A. Lubotzky, R. Phillips, P. Sarnak, *Ramanujan graphs*, Combinatorica **8** (1988) 261–277.
- [19] B. Karrer, M. E. J. Newman, L. Zdeborová, *Percolation on sparse networks*, Phys. Rev. Lett. **113** (2014) 208702.

- [20] N. Alon, I. Benjamini, E. Lubetzky, S. Sodin, *Non-backtracking random walks mix faster*, Commun. Contemp. Math. **9** (2007) 585–603.
- [21] F. Krzakala, C. Moore, E. Mossel, J. Neeman, A. Sly, L. Zdeborová, P. Zhang, *Spectral redemption in clustering sparse networks*, Proc. Natl. Acad. Sci. **110** (2013) 20935–20940.
- [22] M. Karimi, A. H. Banihashemi, *Message-passing algorithms for counting short cycles in a graph*, IEEE Trans. Commun. **61** (2013) 485–495.
- [23] I. F. Blake, S. Lin, *On short cycle enumeration in biregular bipartite graphs*, IEEE Trans. Inform. Theory **64** (2018) 6526–6535.
- [24] A. Dehghan, A. H. Banihashemi, *On computing the multiplicity of cycles in bipartite graphs using the degree distribution and the spectrum of the graph*, IEEE Trans. Inform. Theory **65** (2019) 3778–3789.
- [25] The mathlib community, *The Lean mathematical library*, in: CPP 2020, ACM, 2020, 367–381.
- [26] L. de Moura, S. Ullrich, *The Lean 4 theorem prover and programming language*, in: CADE-28, LNCS 12699, Springer, 2021, 625–635.
- [27] C. Marín, *A machine-checked Godsil–Gutman identity and the Ramanujan band* (Part I of this series), Zenodo, 2026. doi:10.5281/zenodo.20517350.
- [28] C. Marín, *Real roots from matchings: Heilmann–Lieb in Lean 4* (Part II), Zenodo, 2026.
- [29] C. Marín, *Folding edges into vertices: Bass’s determinant formula in Lean 4* (Part III), Zenodo, 2026. doi:10.5281/zenodo.20573120.
- [30] C. Marín, *What a determinant’s derivative knows: Jacobi’s formula and Newton’s identity for matrix traces in Lean 4*, Zenodo, 2026. doi:10.5281/zenodo.20578470.
- [31] C. Marín, *When walks become a spectrum: Godsil’s moment theorem in Lean 4* (Part IV), Zenodo, 2026. doi:10.5281/zenodo.20613246.
- [32] C. Marín, *Counting trees without listing them: Kirchhoff’s matrix-tree theorem in Lean 4* (Part V), Zenodo, 2026. doi:10.5281/zenodo.20629746.